

Self-Attention Mechanics, Positional Encodings, and Scaled Dot-Product Dynamics

A Unified Mathematical, Information-Theoretic, and Physical Foundation of Transformers and Modern Variants

Contents

1	The Transformer Paradigm and Sequence Parallelism	2
2	Scaled Dot-Product & Multi-Head Self-Attention (MHSA)	2
2.1	Mathematical Formulation of Scaled Dot-Product Attention	2
2.2	Mathematical Proof of the $\frac{1}{\sqrt{d_k}}$ Scaling Factor	2
2.3	Multi-Head Self-Attention (MHSA)	3
3	Positional Encodings: Absolute, Relative, and Rotary	4
3.1	Absolute Sinusoidal Positional Encoding	4
3.2	Rotary Position Embedding (RoPE)	4
4	Taxonomy of Transformer Variants	4
4.1	Detailed Analysis of Key Variants	4
4.1.1	1. BERT (Bidirectional Encoder Representations from Transformers)	4
4.1.2	2. GPT Series (Generative Pre-trained Transformer)	5
4.1.3	3. T5 (Text-to-Text Transfer Transformer)	5
4.1.4	4. RoBERTa (Robustly Optimized BERT Approach)	5
4.1.5	5. DeBERTa (Decoding-enhanced BERT with Disentangled Attention)	5
4.1.6	6. Vision Transformer (ViT)	5
4.1.7	7. Swin Transformer (Shifted Windows Transformer)	5
4.1.8	8. Transformer-XL (Extra Long Context)	6
4.1.9	9. Longformer and BigBird	6
4.1.10	10. FlashAttention (Hardware IO-Aware Acceleration)	6
4.1.11	11. Modern Open LLM Architectures (LLaMA / Mistral Frameworks)	6
5	Information-Theoretic and Physical Analogies	6
5.1	Information-Theoretic View: Dynamic Soft Routing	6
5.2	Physical Analogy: Fully-Connected N -Body Particle Mechanics	7
6	Production-Grade PyTorch Implementation	7

1 The Transformer Paradigm and Sequence Parallelism

Prior to the introduction of Transformers (Vaswani et al., 2017), sequence modeling was dominated by Recurrent Neural Networks (RNNs, LSTMs, GRUs) and Convolutional Architectures (CNNs). However, recurrent models suffer from a fundamental architectural limitation: sequential execution along time step t introduces an $\mathcal{O}(T)$ computational dependency chain that prevents parallelization across hardware accelerators (GPUs/TPUs). Furthermore, gradient backpropagation over long sequence lengths T leads to signal attenuation despite gating mechanisms.

****Transformers**** eliminate recurrent hidden states entirely, replacing temporal recurrence with ****Self-Attention Mechanics****. This architectural shift achieves two core advantages: 1. ****Constant Path Length $\mathcal{O}(1)$ **** Any token in a sequence interacts directly with any other token regardless of sequence distance T , bypassing information degradation over long horizons. 2. ****Total Parallelization $\mathcal{O}(1)$ Sequential Operations**** All pairwise token relationships are computed simultaneously using dense matrix multiplications (GEMM).

Definition 1.1 (Sequence Representation Matrix). Let $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}}$ be an input sequence tensor representing T tokens mapped into a d_{model} -dimensional continuous embedding space.

2 Scaled Dot-Product & Multi-Head Self-Attention (MHSA)

Self-attention computes dynamic, data-dependent affinity weights between tokens by projecting input representations into Query ($\mathbf{Q}$), Key ($\mathbf{K}$), and Value ($\mathbf{V}$) spaces.

2.1 Mathematical Formulation of Scaled Dot-Product Attention

Definition 2.1 (Query, Key, and Value Projections). Given input matrix $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}}$, we project inputs using trainable weight matrices $\mathbf{W}_Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $\mathbf{W}_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, and $\mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q \in \mathbb{R}^{T \times d_k}, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K \in \mathbb{R}^{T \times d_k}, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V \in \mathbb{R}^{T \times d_v} \quad (1)$$

Definition 2.2 (Scaled Dot-Product Attention). The attention matrix $\mathbf{A} \in \mathbb{R}^{T \times T}$ and output matrix $\mathbf{O} \in \mathbb{R}^{T \times d_v}$ evaluate as:

$$\mathbf{A} = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} + \mathbf{M} \right) \in \mathbb{R}^{T \times T} \quad (2)$$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathbf{A}\mathbf{V} = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} + \mathbf{M} \right) \mathbf{V} \in \mathbb{R}^{T \times d_v} \quad (3)$$

where $\mathbf{M} \in \mathbb{R}^{T \times T}$ is an optional attention mask matrix (e.g., causal mask $\mathbf{M}_{ij} = -\infty$ for $j > i$, and 0 otherwise).

2.2 Mathematical Proof of the $\frac{1}{\sqrt{d_k}}$ Scaling Factor

Why is the scalar division by $\sqrt{d_k}$ necessary? Without scaling, large key dimensions $d_k \gg 1$ push the dot products $\mathbf{Q}\mathbf{K}^T$ into regions of extremely large magnitude, causing the Softmax function to saturate. Saturating Softmax produces near-zero gradients ($\nabla \text{softmax} \rightarrow \mathbf{0}$), causing vanishing gradients during backpropagation.

Theorem 2.1 (Dot-Product Variance Scaling). Assume query components q_i and key components k_i ($i = 1, 2, \dots, d_k$) are independent random variables with zero mean and unit variance: $\mathbb{E}[q_i] = \mathbb{E}[k_i] = 0$, and $\text{Var}(q_i) = \text{Var}(k_i) = 1$. Then, the variance of their dot product $\mathbf{q} \cdot \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i$ equals d_k :

$$\text{Var}(\mathbf{q} \cdot \mathbf{k}) = d_k \quad (4)$$

Dividing by $\sqrt{d_k}$ normalizes the variance back to 1.

Proof. Evaluate the expectation and variance of the scalar product $z = \mathbf{q} \cdot \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i$:

$$\mathbb{E}[z] = \mathbb{E}\left[\sum_{i=1}^{d_k} q_i k_i\right] = \sum_{i=1}^{d_k} \mathbb{E}[q_i] \mathbb{E}[k_i] = 0 \quad (5)$$

Since q_i and k_i are independent with zero mean:

$$\begin{aligned} \text{Var}(z) &= \mathbb{E}[z^2] - (\mathbb{E}[z])^2 = \mathbb{E}\left[\left(\sum_{i=1}^{d_k} q_i k_i\right) \left(\sum_{j=1}^{d_k} q_j k_j\right)\right] \\ &= \sum_{i=1}^{d_k} \sum_{j=1}^{d_k} \mathbb{E}[q_i q_j k_i k_j] = \sum_{i=1}^{d_k} \mathbb{E}[q_i^2] \mathbb{E}[k_i^2] + \sum_{i \neq j} \underbrace{\mathbb{E}[q_i] \mathbb{E}[q_j] \mathbb{E}[k_i] \mathbb{E}[k_j]}_0 \\ &= \sum_{i=1}^{d_k} (1)(1) = d_k \end{aligned} \quad (6)$$

Because $\text{Var}(z) = d_k$, the standard deviation is $\sqrt{d_k}$. Dividing by $\sqrt{d_k}$ yields $\text{Var}\left(\frac{z}{\sqrt{d_k}}\right) = \frac{\text{Var}(z)}{d_k} = 1$, preserving unit variance across arbitrary embedding dimensions d_k . $\square$

2.3 Multi-Head Self-Attention (MHSA)

Rather than performing a single attention function over d_{model} , Multi-Head Attention projects Queries, Keys, and Values h times in parallel into lower-dimensional projection subspaces $d_k = d_v = d_{\text{model}}/h$. This allows the network to jointly attend to information from different representation subspaces at different positions.

Definition 2.3 (Multi-Head Attention (MHSA)).

$$\text{MHSA}(\mathbf{X}) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h) \mathbf{W}_O \quad (7)$$

$$\text{head}_i = \text{Attention}\left(\mathbf{X} \mathbf{W}_Q^{(i)}, \mathbf{X} \mathbf{W}_K^{(i)}, \mathbf{X} \mathbf{W}_V^{(i)}\right) \in \mathbb{R}^{T \times d_v} \quad (8)$$

where $\mathbf{W}_Q^{(i)} \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $\mathbf{W}_K^{(i)} \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $\mathbf{W}_V^{(i)} \in \mathbb{R}^{d_{\text{model}} \times d_v}$, and $\mathbf{W}_O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

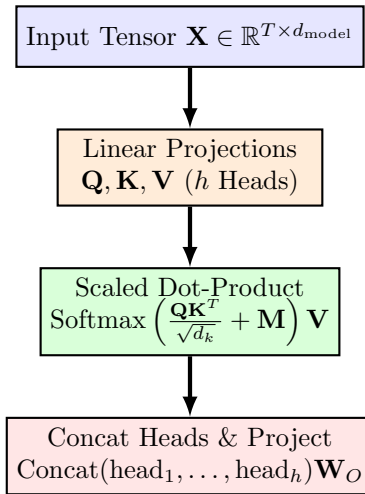


Figure 1: Architecture of Scaled Dot-Product Multi-Head Self-Attention (MHSA).

3 Positional Encodings: Absolute, Relative, and Rotary

Because self-attention is permutation-equivariant ($\text{Attention}(\mathbf{PQ}, \mathbf{PK}, \mathbf{PV}) = \mathbf{P} \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$ for any permutation matrix $\mathbf{P}$), Transformers possess no intrinsic notion of token sequence order. Positional information must be explicitly injected.

3.1 Absolute Sinusoidal Positional Encoding

Vaswani et al. (2017) introduced deterministic, non-learnable sinusoidal encodings added directly to input embeddings $\tilde{\mathbf{X}} = \mathbf{X} + \mathbf{PE}$:

$$\mathbf{PE}_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad \mathbf{PE}_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (9)$$

where $pos \in [0, T-1]$ is token position and $i \in [0, d_{\text{model}}/2-1]$ is dimension index. This formulation allows the model to learn to attend by relative positions, because for any fixed offset k , $\mathbf{PE}_{pos+k}$ can be represented as a linear function of $\mathbf{PE}_{pos}$.

3.2 Rotary Position Embedding (RoPE)

Used in modern Large Language Models (Su et al., 2021; LLaMA, Mistral), **Rotary Position Embedding (RoPE)** incorporates positional information by rotating the Query and Key vectors in complex 2D planes rather than adding static vectors to input embeddings.

Definition 3.1 (Rotary Position Embedding (RoPE)). Let $\mathbf{x}_m \in \mathbb{R}^{d_k}$ be a query or key vector at position m . Group vector elements into 2D pairs $(x_1, x_2), (x_3, x_4), \dots$. RoPE applies a position-dependent rotation matrix $\mathbf{R}_{\Theta, m}^{d_k}$ to each pair:

$$\mathbf{R}_{\Theta, m}^{d_k} \mathbf{x}_m = \begin{pmatrix} \mathbf{R}_{\theta_1, m} & & \mathbf{0} \\ & \ddots & \\ \mathbf{0} & & \mathbf{R}_{\theta_{d_k/2}, m} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_{d_k} \end{pmatrix} \quad (10)$$

where 2D rotation sub-matrices process coordinate pairs with frequency $\theta_i = 10000^{-2(i-1)/d_k}$:

$$\mathbf{R}_{\theta_i, m} = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix} \quad (11)$$

The inner product between rotated query $\mathbf{q}_m = \mathbf{R}_{\Theta, m} \mathbf{q}$ and key $\mathbf{k}_n = \mathbf{R}_{\Theta, n} \mathbf{k}$ depends strictly on their **relative distance** $m - n$:

$$(\mathbf{R}_{\Theta, m} \mathbf{q})^T (\mathbf{R}_{\Theta, n} \mathbf{k}) = \mathbf{q}^T \mathbf{R}_{\Theta, m}^T \mathbf{R}_{\Theta, n} \mathbf{k} = \mathbf{q}^T \mathbf{R}_{\Theta, n-m} \mathbf{k} \quad (12)$$

4 Taxonomy of Transformer Variants

Since Vaswani et al. (2017), the Transformer architecture has undergone extensive evolution across pre-training objectives, attention topology, parameter efficiency, and structural modalities.

4.1 Detailed Analysis of Key Variants

4.1.1 1. BERT (Bidirectional Encoder Representations from Transformers)

BERT (Devlin et al., 2018) utilizes an **Encoder-only** structure. Unlike autoregressive models that read text left-to-right, BERT uses full bidirectional self-attention. It is pre-trained via **Masked Language Modeling (MLM)** (corrupting 15% of input tokens with a '[MASK]' token) and **Next Sentence Prediction (NSP)**.

Variant Name	Structural Paradigm	Attention Mechanism	Key Architectural Innovation
BERT	Encoder-only	Full Bidirectional	Masked LM (MLM) & Next Sentence Pred.
GPT Series	Decoder-only	Causal / Autoregressive	Generative Pre-training & Scale (100B+)
T5	Encoder-Decoder	Full + Causal	Unified Text-to-Text Framework, Relative PE
RoBERTa	Encoder-only	Full Bidirectional	Dynamic Masking, No NSP, Large Batches
DeBERTa	Encoder-only	Disentangled	Disentangled Content/Position Attention
ViT	Encoder-only	Full Spatial	Non-overlapping 16×16 Image Patches
Swin	Hierarchical Encoder	Shifted Local Windows	Localized Window Self-Attention $\mathcal{O}(N)$
Transformer-XL	Recurrent Decoder	Segment Recurrence	State Caching & Relative Positional Enc.
Longformer / BigBird	Encoder-only	Sparse (Global + Local)	Sliding Window + Random + Global Tokens
FlashAttention	Hardware Operator	Exact IO-Aware Tiling	SRAM Tiling, Memory-Aware Recomputation
Modern LLMs (LLaMA/Mistral)	Decoder-only	GQA / MQA + RoPE	Grouped-Query Attn, SwiGLU, RMSNorm

Table 1: Detailed architectural taxonomy comparing major Transformer variants.

4.1.2 2. GPT Series (Generative Pre-trained Transformer)

GPT (Radford et al., 2018, 2019, 2020, 2023) adopts a **Decoder-only** architecture using causal lower-triangular attention masks $\mathbf{M}_{ij} = -\infty \forall j > i$. Trained auto-regressively via standard Next-Token Prediction:

$$\mathcal{L}_{\text{GPT}}(\theta) = - \sum_{i=1}^T \ln P_{\theta}(x_i | x_1, x_2, \dots, x_{i-1}) \quad (13)$$

4.1.3 3. T5 (Text-to-Text Transfer Transformer)

T5 (Raffel et al., 2019) casts all NLP tasks (classification, translation, summarization) into a unified text-to-text format using an **Encoder-Decoder** architecture with relative positional biases added directly to the attention logits.

4.1.4 4. RoBERTa (Robustly Optimized BERT Approach)

RoBERTa (Liu et al., 2019) demonstrated that BERT was severely undertrained. It optimized pre-training by removing the Next Sentence Prediction (NSP) task, introducing **dynamic masking patterns** across training epochs, and scaling mini-batch sizes.

4.1.5 5. DeBERTa (Decoding-enhanced BERT with Disentangled Attention)

DeBERTa (He et al., 2020) improves upon BERT by representing each token using two vectors: a **content vector** $\mathbf{c}_i$ and a **relative position vector** $\mathbf{p}_{i|j}$. Attention scores are disentangled into content-to-content, content-to-position, and position-to-content interactions:

$$A_{i,j} = \mathbf{c}_i \mathbf{c}_j^T + \mathbf{c}_i \mathbf{p}_{j|i}^T + \mathbf{p}_{i|j} \mathbf{c}_j^T \quad (14)$$

4.1.6 6. Vision Transformer (ViT)

ViT (Dosovitskiy et al., 2020) applies pure Transformers directly to image processing. An image $\mathbf{I} \in \mathbb{R}^{H \times W \times C}$ is partitioned into non-overlapping spatial patches of size $P \times P$ (e.g., 16×16). Patches are flattened into vectors $\mathbf{x}_p \in \mathbb{R}^{P^2 C}$, linearly projected into d_{model} dimensions, prepend a learnable ‘[CLS]’ token, and process through standard Transformer encoder blocks.

4.1.7 7. Swin Transformer (Shifted Windows Transformer)

Swin (Liu et al., 2021) constructs a hierarchical vision backbone. Instead of global self-attention $\mathcal{O}((HW)^2)$, Swin evaluates self-attention within **local 2D windows** $\mathcal{O}(HW)$, introducing **Shifted Windowing (SW-MSA)** between layers to allow cross-window communication.

4.1.8 8. Transformer-XL (Extra Long Context)

Transformer-XL (Dai et al., 2019) enables context modeling beyond fixed segment lengths by introducing **Segment-Level Recurrence**. Hidden states computed for previous sequence segments are cached and reused as extended Keys and Values in the current segment, combined with relative positional encodings.

4.1.9 9. Longformer and BigBird

Longformer (Beltagy et al., 2020) and BigBird (Zaheer et al., 2020) reduce the quadratic $\mathcal{O}(T^2)$ self-attention complexity for long documents ($T = 4096+$) by constructing **Sparse Attention Matrices**: combining local sliding window attention ($\mathcal{O}(T \cdot w)$), random sparse connections, and dedicated global memory tokens.

4.1.10 10. FlashAttention (Hardware IO-Aware Acceleration)

FlashAttention (Dao et al., 2022, 2023) is a memory-efficient algorithm that speeds up exact self-attention without approximation. Classical attention writes intermediate $T \times T$ matrices to high-bandwidth GPU HBM memory. FlashAttention uses **Tiling** and **Online Softmax Normalization** to perform block-by-block computation strictly within ultra-fast GPU SRAM cache, reducing memory IO reads/writes from $\mathcal{O}(T^2)$ to $\mathcal{O}(T)$.

4.1.11 11. Modern Open LLM Architectures (LLaMA / Mistral Frameworks)

Modern state-of-the-art Open LLMs incorporate three core structural improvements: **Grouped-Query Attention (GQA) / Multi-Query Attention (MQA)**: Instead of assigning 1 Key/Value head per Query head ($h_K = h_Q$), GQA maps groups of g Query heads to a single shared Key/Value head ($h_K = h_Q/g$). This reduces Key-Value Cache (KV-Cache) memory footprint during inference by $g \times$. **SwiGLU Activation Function**: Replaces traditional ReLU/GELU in feed-forward sub-layers with the **SwiGLU** (Swish Gated Linear Unit; Shazeer, 2020) activation:

$$\text{SwiGLU}(\mathbf{x}) = (\mathbf{x}\mathbf{W}_1 \odot \text{Swish}(\mathbf{x}\mathbf{W}_2)) \mathbf{W}_3, \quad \text{Swish}(z) = z \cdot \sigma(\beta z) \quad (15)$$

RMSNorm (Root Mean Square Normalization): Replaces standard Layer Normalization (LayerNorm) by removing mean-centering, scaling strictly by the root mean square of feature activations (Zhang & Sennrich, 2019):

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \odot \gamma, \quad \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon} \quad (16)$$

5 Information-Theoretic and Physical Analogies

5.1 Information-Theoretic View: Dynamic Soft Routing

In information theory, self-attention performs **Dynamic Soft Routing** over continuous information channels.

Given input tokens $\mathbf{X}$, the Query-Key inner product matrix $\mathbf{QK}^T/\sqrt{d_k}$ computes the pairwise **Mutual Information Affinity** between sequence positions. Softmax normalization yields conditional probability routing distributions $P(\text{Token}_j | \text{Token}_i)$.

Applying attention $\mathbf{AV}$ minimizes conditional cross-entropy $H(\mathbf{Y} | \mathbf{X})$ by dynamically routing context from informative source tokens V_j to target request tokens Q_i .

5.2 Physical Analogy: Fully-Connected N -Body Particle Mechanics

Self-attention maps onto a classical or quantum **Fully-Connected N -Body Interacting Particle Field**.

Consider T interacting particles in d_{model} -dimensional space:

- Tokens represent physical particles with spatial state vectors $\mathbf{x}_i$.
- Queries and Keys define anisotropic physical interaction potentials $U(\mathbf{x}_i, \mathbf{x}_j) = -\frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$.
- Attention weights $\mathbf{A}_{ij} = \frac{\exp(-U(\mathbf{x}_i, \mathbf{x}_j))}{\sum_k \exp(-U(\mathbf{x}_i, \mathbf{x}_k))}$ act as normalized **Boltzmann-Gibbs Thermal Interaction Coefficients**.
- Output values $\mathbf{V}_j$ represent local force field contributions. Self-attention calculates the mean-field equilibrium state of an N -body physical system.

6 Production-Grade PyTorch Implementation

The following Python code provides a complete, production-grade implementation of a Modern Transformer Decoder Block built in PyTorch. It features **Rotary Position Embeddings (RoPE)**, **Grouped-Query Attention (GQA)**, **SwiGLU Feed-Forward Networks**, **RMSNorm**, and Causal Attention Masking.

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 class RMSNorm(nn.Module):
6     """Root Mean Square Layer Normalization (Zhang & Sennrich, 2019)."""
7     def __init__(self, dim: int, eps: float = 1e-6):
8         super().__init__()
9         self.eps = eps
10        self.weight = nn.Parameter(torch.ones(dim))
11
12    def forward(self, x: torch.Tensor) -> torch.Tensor:
13        # RMS = sqrt(mean(x^2) + eps)
14        rms = torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps)
15        return x * rms * self.weight
16
17
18 class RotaryPositionEmbedding(nn.Module):
19     """Rotary Position Embedding (RoPE) for relative position injection."""
20     def __init__(self, dim: int, max_seq_len: int = 4096, theta: float = 10000.0):
21         super().__init__()
22         self.dim = dim
23         # Compute frequencies: theta_i = theta^(-2(i-1)/dim)
24         freqs = 1.0 / (theta ** (torch.arange(0, dim, 2).float() / dim))
25         t = torch.arange(max_seq_len).float()
26         freqs_matrix = torch.outer(t, freqs) # (max_seq_len, dim/2)
27
28         # Cache cos and sin embeddings
29         self.register_buffer("cos_cached", torch.cos(freqs_matrix)) # (
max_seq_len, dim/2)
30         self.register_buffer("sin_cached", torch.sin(freqs_matrix))
31
32    def _rotate_half(self, x: torch.Tensor) -> torch.Tensor:
33        """Rotates 2D vector pairs: [-x2, x1]."""
34        x1 = x[..., :self.dim // 2]
```

```

35         x2 = x[..., self.dim // 2:]
36         return torch.cat((-x2, x1), dim=-1)
37
38     def forward(self, x: torch.Tensor, seq_len: int) -> torch.Tensor:
39         # x shape: (batch_size, num_heads, seq_len, head_dim)
40         cos = self.cos_cached[:seq_len, :].unsqueeze(0).unsqueeze(1) # (1,
1, seq_len, head_dim/2)
41         sin = self.sin_cached[:seq_len, :].unsqueeze(0).unsqueeze(1)
42
43         # Tile cos/sin across full head_dim
44         cos = torch.cat((cos, cos), dim=-1)
45         sin = torch.cat((sin, sin), dim=-1)
46
47         return (x * cos) + (self._rotate_half(x) * sin)
48
49
50 class GroupedQueryAttention(nn.Module):
51     """
52     Grouped-Query Attention (GQA) supporting MHA, MQA, and GQA
53     configurations,
54     integrated with RoPE and Causal Masking.
55     """
56     def __init__(self, d_model: int, num_heads_q: int = 8, num_heads_kv:
int = 2):
57         super().__init__()
58         self.d_model = d_model
59         self.num_heads_q = num_heads_q
60         self.num_heads_kv = num_heads_kv
61         self.num_groups = num_heads_q // num_heads_kv # Expansion ratio for
KV heads
62         self.head_dim = d_model // num_heads_q
63
64         self.q_proj = nn.Linear(d_model, num_heads_q * self.head_dim, bias=
False)
65         self.k_proj = nn.Linear(d_model, num_heads_kv * self.head_dim, bias
=False)
66         self.v_proj = nn.Linear(d_model, num_heads_kv * self.head_dim, bias
=False)
67         self.out_proj = nn.Linear(num_heads_q * self.head_dim, d_model,
bias=False)
68
69         self.rope = RotaryPositionEmbedding(dim=self.head_dim)
70
71     def forward(self, x: torch.Tensor) -> torch.Tensor:
72         B, T, C = x.shape
73
74         # 1. Project Queries, Keys, and Values
75         q = self.q_proj(x).view(B, T, self.num_heads_q, self.head_dim).
transpose(1, 2)
76         k = self.k_proj(x).view(B, T, self.num_heads_kv, self.head_dim).
transpose(1, 2)
77         v = self.v_proj(x).view(B, T, self.num_heads_kv, self.head_dim).
transpose(1, 2)
78
79         # 2. Apply Rotary Position Embeddings (RoPE)
80         q = self.rope(q, seq_len=T)
81         k = self.rope(k, seq_len=T)
82
83         # 3. Expand KV heads to match Q heads for Grouped-Query Attention
84         if self.num_groups > 1:

```



```

84         k = k.repeat_interleave(self.num_groups, dim=1) # (B,
num_heads_q, T, head_dim)
85         v = v.repeat_interleave(self.num_groups, dim=1)
86
87         # 4. Scaled Dot-Product Attention Scores
88         scores = (q @ k.transpose(-2, -1)) / (self.head_dim ** 0.5) # (B,
num_heads_q, T, T)
89
90         # 5. Apply Causal Lower-Triangular Mask
91         causal_mask = torch.triu(torch.full((T, T), float('-inf')), device=x
.device), diagonal=1)
92         scores = scores + causal_mask.unsqueeze(0).unsqueeze(1)
93
94         attn_weights = F.softmax(scores, dim=-1)
95         out = attn_weights @ v # (B, num_heads_q, T, head_dim)
96
97         # 6. Concatenate Heads and Final Output Projection
98         out = out.transpose(1, 2).contiguous().view(B, T, -1)
99         return self.out_proj(out)
100
101
102 class SwiGLUFFN(nn.Module):
103     """Swish Gated Linear Unit (SwiGLU) Feed-Forward Network."""
104     def __init__(self, d_model: int, hidden_dim: int):
105         super().__init__()
106         self.w1 = nn.Linear(d_model, hidden_dim, bias=False)
107         self.w2 = nn.Linear(d_model, hidden_dim, bias=False)
108         self.w3 = nn.Linear(hidden_dim, d_model, bias=False)
109
110     def forward(self, x: torch.Tensor) -> torch.Tensor:
111         # SwiGLU(x) = (w1(x) * Swish(w2(x))) @ w3
112         return self.w3(F.silu(self.w2(x)) * self.w1(x))
113
114
115 class ModernTransformerBlock(nn.Module):
116     """Modern Transformer Decoder Block combining RMSNorm, GQA, RoPE, and
SwiGLU."""
117     def __init__(self, d_model: int = 512, num_heads_q: int = 8,
num_heads_kv: int = 2):
118         super().__init__()
119         self.attn_norm = RMSNorm(d_model)
120         self.attn = GroupedQueryAttention(d_model, num_heads_q,
num_heads_kv)
121
122         self.ffn_norm = RMSNorm(d_model)
123         # Standard hidden dimension expansion (8/3 * d_model)
124         hidden_dim = int(2 * (4 * d_model) / 3)
125         self.ffn = SwiGLUFFN(d_model, hidden_dim)
126
127     def forward(self, x: torch.Tensor) -> torch.Tensor:
128         # Pre-LN / Pre-RMSNorm Residual Connection
129         x = x + self.attn(self.attn_norm(x))
130         x = x + self.ffn(self.ffn_norm(x))
131         return x
132
133
134 if __name__ == "__main__":
135     torch.manual_seed(42)
136
137     # Instantiate Modern Transformer Decoder Block

```

```

138     batch_size = 2
139     seq_length = 16
140     embedding_dim = 256
141
142     x_input = torch.randn(batch_size, seq_length, embedding_dim)
143
144     transformer_block = ModernTransformerBlock(
145         d_model=embedding_dim,
146         num_heads_q=8,
147         num_heads_kv=2
148     )
149
150     output_tensor = transformer_block(x_input)
151
152     print("=== MODERN TRANSFORMER BLOCK INITIALIZATION SUCCESSFUL ===")
153     print(f"Input Tensor Shape : {x_input.shape} (Batch, Seq_Len, d_model)"
154           ")")
154     print(f"Output Tensor Shape : {output_tensor.shape} (Batch, Seq_Len,
155           d_model)")
155     print(f"Total Block Parameters: {sum(p.numel() for p in
156           transformer_block.parameters())}")

```

Listing 1: Production-grade PyTorch implementation of a Modern Transformer Block with GQA, RoPE, and SwiGLU.